

Why Backend & Introduction to Node.js & Express.js

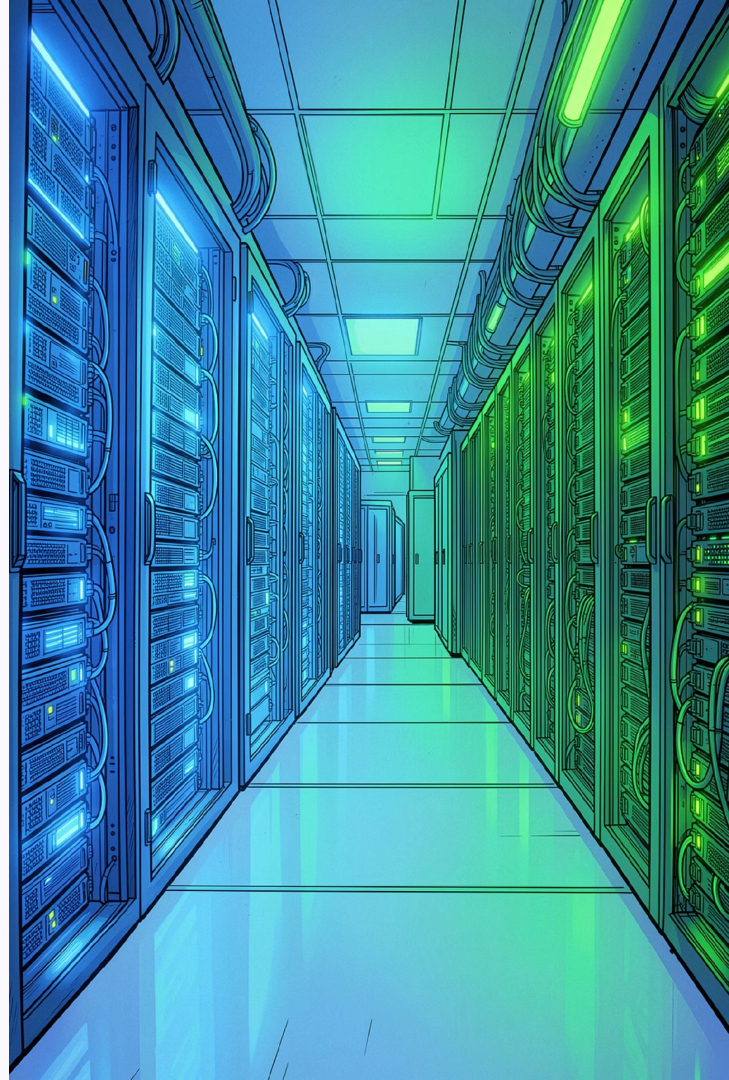
TITANS-SUKKUR

SIR YASIR ALI LASHARI

BACKEND DEVELOPMENT

NODE.JS

EXPRESS



What Is Backend?

Every app has two parts: the **frontend** (what users see) and the **backend** (the engine behind the scenes). The backend also talks to a **database** to store and retrieve data.

1

Frontend

Mobile app sends a request

2

Backend Server

Validates user, processes logic, stores data

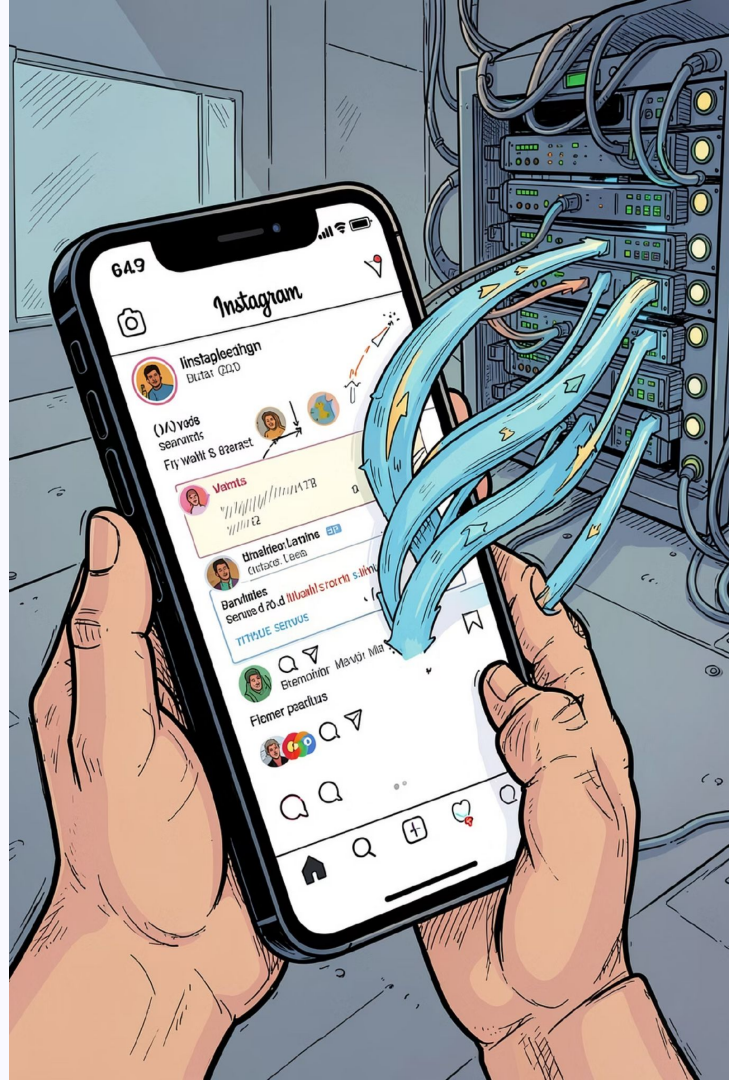
3

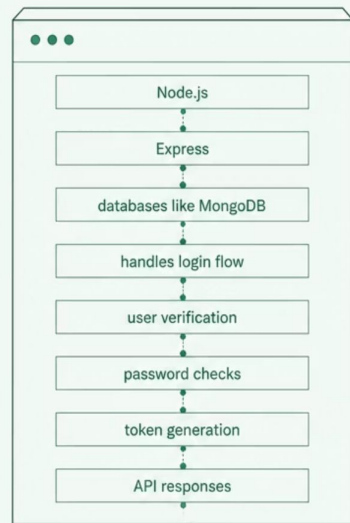
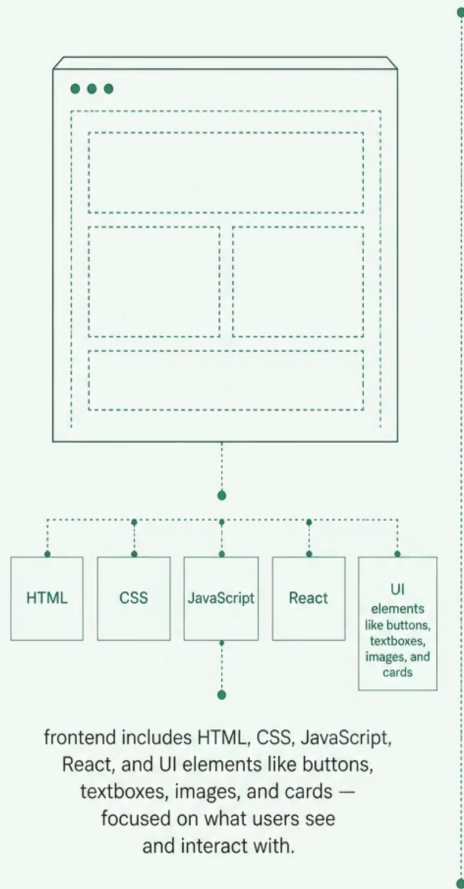
Database

Persists data and returns it on demand



Example — Instagram photo upload: App → Backend validates → Image stored → Data saved to DB → Response sent back.





backend includes Node.js, Express, databases like MongoDB, and handles login flow, user verification, password checks, token generation, and API responses — focused on business logic and data.

FRONTEND VS. BACKEND

The **frontend** is everything the user interacts with — buttons, forms, images, and layouts.

The **backend** is the invisible layer that processes requests, enforces rules, and manages data.

- Frontend = UI & user experience
- Backend = logic, security & data
- Both communicate through APIs



Why Backend? Four Core Responsibilities



Authentication

Verify usernames, check passwords, and issue secure tokens on login.



Database Management

Save, retrieve, update, and delete data safely using a database like MongoDB.



Security

Protect passwords, payments, and user data – logic that must never be exposed on the frontend.

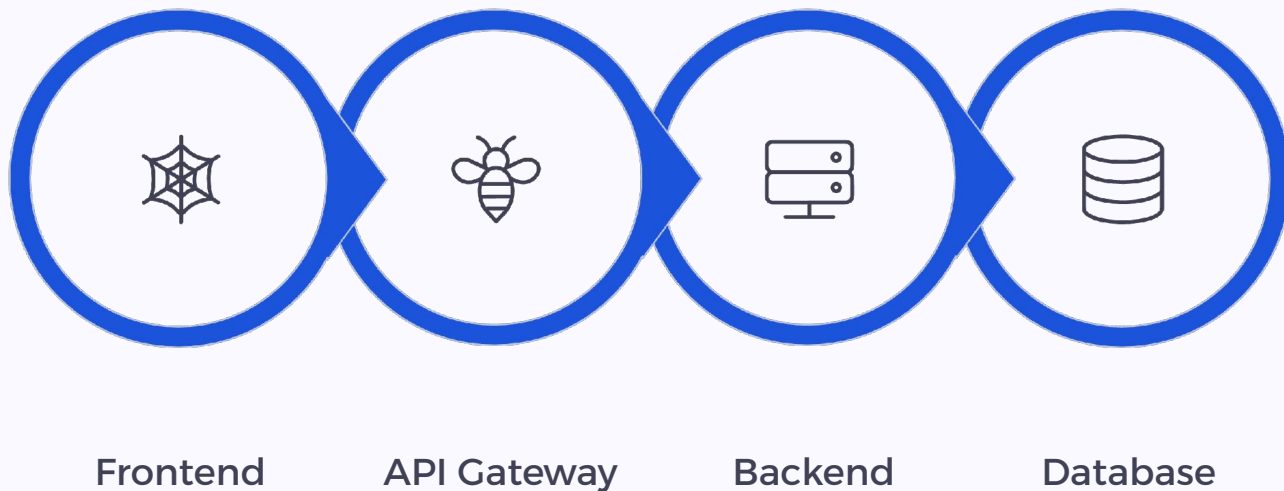


Business Logic

Handle complex flows like checking stock, calculating tax, and processing payments (e.g., Amazon).

How APIs Connect Everything

The frontend **never** talks directly to the database. All communication flows through an **API** – a controlled gateway managed by the backend.



This architecture keeps your data secure, your logic centralized, and your frontend free to focus on the user experience.

What Is Node.js?

Node.js is a **runtime environment** that allows JavaScript to run **outside the browser** – on a server, in the cloud, or in your terminal.

In the Browser

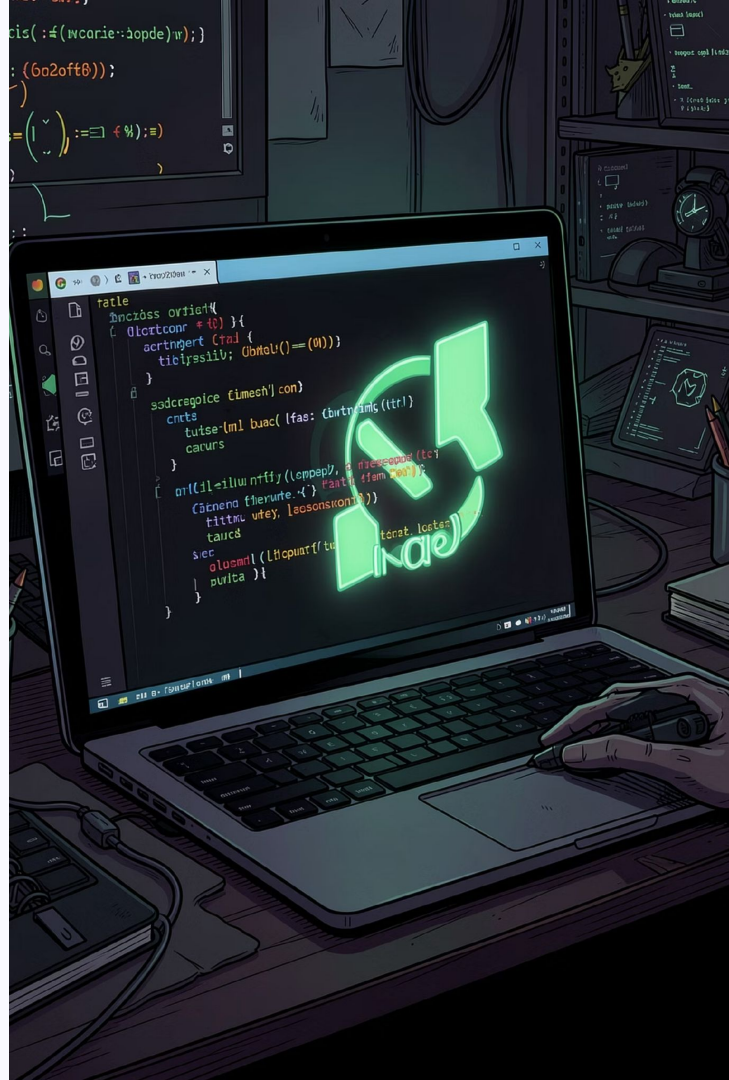
JavaScript runs inside Chrome, Firefox, or Edge. It controls the DOM and responds to user clicks.

```
alert("Hello");
```

With Node.js

JavaScript runs on a server or computer. It can read files, connect to databases, and run an HTTP server.

```
console.log("Hello");
```



Why JavaScript for Backend?

Before Node.js

Developers had to juggle multiple languages:

- Frontend → JavaScript
- Backend → Java, PHP, Python, or C#

Context-switching between languages slowed teams down and increased complexity.

After Node.js

One language powers the entire stack:

- Frontend → JavaScript
- Backend → JavaScript (Node.js)

Same syntax, same ecosystem, same tooling – from browser to server. **JavaScript everywhere.**

Who Uses Node.js?

Node.js powers some of the world's most demanding applications – from streaming to payments to ride-sharing.



Netflix

Uses Node.js for its UI layer, serving millions of streams globally



PayPal

Migrated from Java to Node.js, cutting development time significantly



Uber

Relies on Node.js for its real-time dispatch and mapping systems



LinkedIn

One of the earliest adopters, using Node.js for its mobile backend



Walmart & eBay

Handle massive traffic spikes during sales events with Node.js

What is Express.js?

Express.js is a **web application framework** built on top of Node.js.

The Engine (Node.js)

Node.js provides the **raw power** to create servers and handle network requests. It's the core component, enabling JavaScript to run on the backend. Building a full web application with just Node.js would be like driving a car with only an engine - powerful, but **extremely difficult** and requiring a lot of custom work.

The Car (Express.js)

Express.js is the **complete car**. It takes the raw power of Node.js and provides ready-made tools, middleware, and a clear structure. This simplifies the development process, making it much easier to build scalable web applications and APIs with **less repetitive code**.

What You'll Build

By the end, you'll have hands-on experience building real-world backend systems.



Auth System

Secure login, signup, and token-based authentication



Blog API

Full CRUD API for posts, comments, and users



E-commerce Backend

Products, orders, and inventory management



Chat Application

Real-time messaging backend with WebSockets



File Upload API

Secure file handling and storage



Payment Integration

Process payments and handle transactions



REST APIs

Production-ready, scalable RESTful services